PyCHC: a Framework for Certified Horn Solving and CHC-based Design

Anna Becchi[1]✉, Martin Blicha[2], Rodrigo Otoni[3], and Natasha Sharygina[1]

[1] University of Lugano, Lugano, Switzerland {beccha,sharygin}@usi.ch
[2] Argot Collective, Zug, Switzerland martin.blicha@argot.org
[3] University of Groningen, Groningen, Netherlands r.b.otoni@rug.nl

To appear in the proceedings of CAV'26 (https://conferences.i-cav.org/2026)

Abstract. We present PyCHC, a solver-agnostic framework aimed at systems of constrained Horn clauses (CHC). PyCHC provides intuitive Python APIs to create and manipulate CHC systems programmatically, and solve them using different backend solvers. Furthermore, PyCHC offers a certification pipeline to validate the correctness of results reported by the CHC solvers, via the use of independent satisfiability modulo theories (SMT) solvers and proof checkers. We present our framework's architecture and features, and demonstrate how it enables rapid prototyping of new CHC-based algorithms and experimentation with novel strategies for cooperative solving. We used PyCHC to validate the results of the Eldarica, Golem, and Z3-Spacer solvers on CHC-COMP benchmarks, finding several issues across different tool versions.

Keywords: Constrained Horn Clauses · SMT · Certified Solving

1 Introduction

Systems of constrained Horn clauses (CHC) [16] represent an important fragment of first-order logic, which has seen much use in software verification [6], e.g., analysing programs written in C/C++ [17], Java [26], Rust [29], and Solidity [35], as well as synthesis [27,37], parameterised systems [18], and hyperproperties [24] verification. To automate their use, many CHC solvers have been developed, such as Eldarica [22], FreqHorn [14], Golem [7], HoIce [10], MuCyc [42] and Z3-Spacer [28].

CHC solvers are developed on large and complex codebases, and are themselves not exempt from bugs. Seeing state-of-the-art solvers disagree on certain benchmarks in the annual CHC solving competition, CHC-COMP, made the validation of their results an important concern of the competition organisers [13]. Certification has been an active research area, especially in the context of Boolean satisfiability (SAT) [40,20,19,44,12,2] and satisfiability modulo theories (SMT) [30,41,32,39,21], although focused efforts on CHC solving have

emerged only in recent years. In our previous work, we introduced two complementary methods for certifying CHC satisfiable [33] and unsatisfiable [34] results, which have been respectively implemented in the ATHENA framework[4] and the Golem solver.[5] The essence of these approaches is to define a certification pipeline that leverages the production of witnesses and their cross-validation with independent tools, thereby strengthening the confidence in the correctness of the results. Unfortunately, while SMT-LIB is a well-established standard for representing CHC satisfiable witnesses, there is no unifying standard for proof production and certification of CHC unsatisfiable results. To the best of our knowledge, the approach presented in [34] is the only method proposed for certifying unsatisfiable CHC results, and the Golem solver remains the only tool that implements it. Moreover, there is no unifying framework that integrates the efforts from [33] and [34] and that is capable of certifying both satisfiable and unsatisfiable CHC results.

Besides the ability to ensure the correctness of solving results, two other desirable features for CHC usage are the possibility to employ different solvers seamlessly and to prototype novel solving algorithms easily. These features proved beneficial in the context of SMT solvers, and were made possible by tools such as JavaSMT [3], SMT.ml [36], and PySMT [15].

With the aim of providing a unified tool that facilitates CHC usage and algorithm prototyping, and also strengthens correctness guarantees, we present PyCHC,[6] a framework for certified Horn solving and CHC-based design. Our framework provides intuitive Python APIs to create CHC systems programmatically, read them from files, and inspect and modify their clauses. The satisfiability of a CHC system can be checked using different backend solvers, thanks to common interfaces that abstract away tool-specific details. One of the most important features offered by PyCHC is a solver-agnostic certification pipeline for CHC solving. Satisfiable CHC answers are validated using a two-layered approach involving SMT solving and proof checking, while unsatisfiable CHC answers are validated using a proof checker, thereby effectively combining the techniques from [33] and [34]. In PyCHC, certification can run automatically after solving, or be invoked to check externally produced results. A by-product of this pipeline is that our framework also allows for the validation of unsatisfiability of a given SMT formula via proof checking.

PyCHC natively supports three CHC solvers (Eldarica [22], Golem [7], and Z3-Spacer [28]), three SMT solvers (cvc5 [4], OpenSMT [9,23], and Z3 [31]), and one proof checker (Carcara [1]). Besides these tools, more solvers and checkers can be easily added, due to PyCHC's modular architecture. For the time being, PyCHC's support for unsatisfiable CHC certification is limited to the Golem solver, which is the only tool that supports proof production [34]. Our hope is that a unifying framework like PyCHC serves as an encouragement

[4] ATHENA is available at github.com/usi-verification-and-security/athena.
[5] Golem is available at github.com/usi-verification-and-security/golem.
[6] PyCHC is available at github.com/usi-verification-and-security/pychc.

for the community to adopt a standard for unsatisfiable CHC certification, and that more solvers will support proof production in the future.

We provide several examples demonstrating how PyCHC can be used beyond plain CHC solving. By introducing an abstraction layer on top of different CHC and SMT solvers, it enables rapid prototyping of new algorithms that reduce to, or leverage, CHC solving (e.g., liveness/termination checking). This abstraction layer also allows for experimentation with strategies for cooperative solving (e.g., portfolio solving, and invariant sharing when verifying multiple properties).

We used PyCHC to certify the results produced by Eldarica, Golem, and Z3-Spacer on benchmarks from CHC-COMP, and found several issues in these mature solvers. We also demonstrate how PyCHC can be used to track the status of these issues across different tool versions.

The remainder of this paper is structured as follows: Section 2 discusses related work; Section 3 provides the necessary notation and background on CHC systems; Section 4 describes the architecture and main features of PyCHC; Section 5 reports on the usage of PyCHC on the CHC-COMP benchmarks and the issues found; Section 6 concludes the paper and outlines future work.

2 Related Work

Interest in certified solving started over a decade ago in the context of SAT solvers, and has since expanded to SMT and CHC solvers.

For SAT, a satisfying assignment can be certified simply by substituting the variables with their interpretations and checking if the obtained formula evaluates to *true*. Certification of unsatisfiability, on the other hand, is far from trivial, with approaches based on resolution [40], clausal proofs [20], and resolution asymmetric tautology (RAT) [19,44,12,2] being proposed.

In the SMT case, certification is complicated by the presence of theories and quantifiers. Satisfying assignments currently have no uniform format in line with the SMT-LIB standard [5], which presents an obstacle for certification, with efforts being made in this direction by organisers of the SMT competition [25]. Regarding unsatisfiability results, different approaches have been proposed, such as the logical framework with side conditions (LFSC) [41], supported by cvc5, the Alethe proof format [39], supported by cvc5 and veriT [8], and unnamed approaches incorporated into SMTInterpol [21], OpenSMT [32], and Z3 [30].

When it comes to CHC solving, efforts are more recent. CHC solvers such as Eldarica, Golem, and Z3-Spacer have internal mechanisms to check the produced results. However, internal validation is generally opaque and solver-specific: being tightly coupled with the solver's implementation, it cannot provide a strong guarantee of correctness. Moreover, post-processing (e.g., reverting transformations applied to the original CHC system) and output phases of the solver can be buggy, and an incorrect result can be returned to the user despite internal validations. For these reasons, an external cross-solvers certification of the produced outputs is desirable.

An initial approach for external certification of satisfiable results was implemented via an ad-hoc validator tied to the SMT solver Z3.[7] For a more solver-agnostic approach, we leveraged the established SMT-LIB standard supported by both CHC and SMT solvers in the ATHENA framework [33]: ATHENA is a collection of scripts orchestrating the invocation of the solvers and proof checkers, redirecting their output to the next tool in the pipeline with minor syntactic manipulations. ATHENA enabled a systematic satisfiability certification with independent tools and spotted several issues in mature solvers; however, it has a limited support for an interactive debugging and investigation of the produced models, and its design is not easily extensible to support new solvers and proof formats. For the certification of unsatisfiable results, we developed a method relying on the generation of externally checkable proofs in [34]. So far, the Golem CHC solver is the only tool that supports production of Alethe proofs for unsatisfiable results, which can be independently validated by the Carcara proof checker [1]. PyCHC extends the pipeline introduced by ATHENA with the certification of unsatisfiable results, as pioneered by Golem. PyCHC significantly generalises such former approaches in a unified library with easily extendible interfaces and several new features, including the programmatic investigation of the produced witnesses – useful when comparing the results of different solvers on the same benchmark.

PyCHC can also be used beyond certification tasks. We took inspiration from the solver-agnostic library PySMT [15], which offers uniform APIs to programmatically interact with multiple SMT solvers and manipulate SMT formulae. Due to its flexibility and ease of use, PySMT has been successfully employed in teaching and in the prototyping of new SMT-based algorithms and tools. For example, PySMT can be used to quickly sketch algorithms such as BMC, k-induction, and IC3,[8] helping in the understanding of high-level concepts and the exploration of new ideas. PyCHC lifts PySMT's idea to the CHC domain, offering abstractions and modelling support specific for CHC systems. With PyCHC it is possible to re-implement existing algorithms that reduce to CHC solving, and experiment with new ones. A similar abstraction is offered by PyVMT[9] for safety and liveness model checking, although it does not provide certified solving capabilities to our knowledge.

3 Preliminaries

Given a first-order theory $\mathcal{T}$ and a set of uninterpreted predicates $\mathcal{P}$ disjoint from the signature of $\mathcal{T}$, a CHC is an implication of form $\varphi \wedge P_1 \wedge \ldots \wedge P_n \rightarrow H$, where φ is an interpreted formula in the language of $\mathcal{T}$, and H and every P_i are applications of a predicate symbol in $\mathcal{P}$ to terms of $\mathcal{T}$. All variables in the clause are implicitly universally quantified. The antecedent and consequent of

[7] See github.com/chc-comp/chc-tools/blob/master/chctools/chcmodel.py.
[8] See pysmt.readthedocs.io/en/latest/tutorials.html.
[9] See github.com/pyvmt/pyvmt.

the clause are called *body* and *head*, while φ is its *constraint*. A clause is called *query* if its head is *false*, and a *fact* if its body has no uninterpreted predicates.

A set of constrained Horn clauses is called a CHC *system*. A given system $\mathcal{S}$ is *satisfiable* if there exists a model $\mathcal{M}$ of $\mathcal{T}$ extended with an interpretation for the uninterpreted predicates $\mathcal{P}$ s.t. $\mathcal{M} \models c$, for every clause $c \in \mathcal{S}$ [38]. Satisfiability can be witnessed by an *interpretation*, a mapping from the uninterpreted predicates $\mathcal{P}$ to formulas in the language of $\mathcal{T}$, such that every clause becomes a valid formula in $\mathcal{T}$ after the predicates are replaced by their interpretations. The unsatisfiability of $\mathcal{S}$ is witnessed by a derivation of *false* in some sound proof system, typically based on instantiation and (hyper-)resolution.

4 The PyCHC Framework

In this section, we present the PyCHC framework, the first tool offering solver-agnostic robust certified CHC solving support, along with easy-to-use modelling and prototyping functionalities. We first describe the tool's architecture, in Section 4.1, and then detail the certified solving workflow, in Section 4.2, and the modelling support, in Section 4.3.

4.1 Tool Architecture

The architecture of PyCHC is illustrated in Figure 1. The framework is implemented in Python, with a modular design. At the top level, PyCHC exposes APIs for (i) creating and manipulating CHC systems and models, (ii) invoking CHC or SMT solvers, and (iii) orchestrating the certification pipeline.

A CHC system can be created from scratch, by adding/removing predicates and clauses one by one, or read from an SMT-LIB file. PyCHC adds an abstraction layer on top of the PySMT library to construct, syntactically manipulate, and inspect formulae, predicates, clauses, and models, as well as for printing and parsing CHC systems in the SMT-LIB format.

Abstract interfaces for CHC solvers, SMT solvers, and proof checkers (PC in Figure 1) provide general APIs to interact with external tools. Concrete solvers can be integrated by providing the path to their executables, and specifying

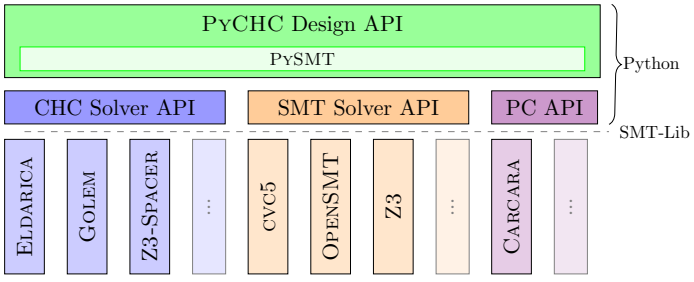


Fig. 1. Architecture of the PyCHC framework.

```
1  chc_system = CHCSystem.load_from_file("file.smt2")
2
3  # Solve and certify the result (both sat or unsat)
4  golem = GolemSolver(smt_validator=OpenSMTSolver(),proof_checker=Carcara())
5  golem.load_system(chc_system)
6  status = golem.solve(validate=True)
7
8  # Certify an externally provided CHC model
9  chc_system.validate_sat_model(
10     witness=SatWitness.load_from_file("witness.smt2"),
11     smt_validator=CVC5Solver(proof_checker=Carcara()),
12     )
13
14 # Certify unsat answer of SMT solver
15 f = SMTLibFormula.load_from_file("formula.smt2")
16 smtsolver = CVC5Solver(proof_checker=Carcara(), logic=get_logic(f))
17 if not smtsolver.is_sat(f):
18     smtsolver.validate_proof()
```

Fig. 2. Code snippet illustrating how to invoke the certification pipeline in PyCHC.

the command-line arguments needed to invoke common functionalities. In this way, PyCHC can programmatically toggle options such as witness generation, without exposing the user to the specificities of each tool. The solvers execute as subprocesses, and communication is handled via the SMT-LIB format and standard input/output streams. Such architectural choice allows for a non-intrusive and reproducible invocation of any CHC solver, most of which do not offer APIs for direct integration.

Currently, PyCHC integrates Eldarica, Golem and Z3-Spacer as CHC solvers; cvc5, OpenSMT, and Z3 as SMT solvers; and Carcara as a proof checker. Commonly used options (such as changing the backend *engine* in Golem, or toggling *global guidance* in Z3-Spacer [43]) are already made available via parameters in the respective solver classes. Different releases of the same tool can be easily integrated by specifying different executable paths.

4.2 Certified CHC Solving

The core goal of PyCHC is to provide a uniform access to CHC solvers and certify their answers with independent tools. Certification can be invoked in several ways, three of which are illustrated in Figure 2. A CHC solver can be accompanied by an SMT solver and/or a proof checker. For example, in line 4, the concrete CHC solver Golem is instantiated, and equipped with OpenSMT as SMT solver and Carcara as proof checker. In line 6, a CHC system that was previously loaded from a SMT-LIB file is solved and the produced answer is automatically certified using the specified tools. It is then possible to ask the solver for the solving status, or the produced witness. One can also certify results computed by external CHC solvers. As shown in lines 9–12, a (candidate) interpretation for the predicates can be read from a file and validated against the CHC system using a specified SMT solver and proof checker. A similar approach can be used to read and certify an unsatisfiable answer produced by an external CHC solver. SMT solving results can be certified as well (in case of

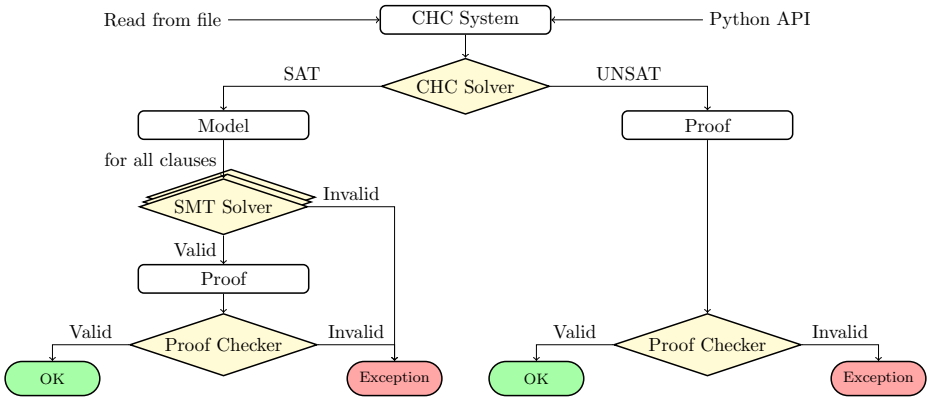


Fig. 3. Certification pipeline for CHC solving in PyCHC.

unsatisfiability): lines 16–18 illustrate how to check the satisfiability of a formula and validate the proof produced by the SMT solver, if any.

For both CHC and SMT solvers, it is possible to execute a specific SMT-LIB file without passing through PySMT's parser and printers, thus checking the solver's behaviour exactly on the provided input file. If the file contains a `(get-model)` or `(get-proof)` command, PyCHC will automatically store the produced witness for further certification. After reading the input file, the SMT solver maintains its interactive solving session open, therefore allowing to add assertions and send new commands. Both this functionality and the certification of SMT unsatisfiability proofs are novel features that are not present in PySMT.

Whenever a validation is not successful, an exception is raised informing the user that the provided witness is invalid. In this case, the SMT-LIB files representing the failing queries are saved to disk for further investigation.

Certifying a CHC satisfiable answer. For validating a satisfiable CHC answer, PyCHC follows the two-layered approach originally presented in [33]. The flow is illustrated by the left-hand side branch of Figure 3.

Let $\mathcal{M}$ be the model returned by a CHC solver. As a first step, an SMT solver checks the validity of each Horn clause in the CHC system, where each uninterpreted predicate P is replaced by $\mathcal{M}(P)$, its interpretation in $\mathcal{M}$. Formally, for each clause $c \equiv \varphi \wedge P_1 \wedge \ldots \wedge P_n \rightarrow H$, an SMT solver checks the satisfiability of the formula:

$$\varphi \wedge \mathcal{M}(P_1) \wedge \ldots \wedge \mathcal{M}(P_n) \wedge \neg \mathcal{M}(H) \tag{1}$$

If formula (1) is satisfiable then the model is invalid, as it falsifies that clause. Otherwise, PyCHC continues with a second step of certification. The SMT solver is asked to produce a proof of unsatisfiability of formula (1), i.e., a certificate that $\mathcal{M} \models c$, and a proof checker is used to validate it.

Certifying a CHC unsatisfiable answer. When the CHC solver returns UNSAT (shorthand for unsatisfiable), it means that there is no interpretation for the predicates that satisfies all the clauses: in this case, a CHC solver also produces a proof of unsatisfiability which consists of a sequence of variable assignments and clause applications leading to a contradiction. In [34], it is shown how a CHC solver can be extended to provide such resolution-style proofs in a certifiable format. The flow is illustrated by the right-hand side branch of Figure 3.

As an example, the Golem CHC solver is able to produce Alethe proofs, which can be independently checked by the Carcara proof checker. Although UNSAT certification is currently limited to Golem, the modular interface of PyCHC easily allows for the integration of certification for other CHC solvers, as they introduce equivalent proof generation capabilities.

4.3 Design Environment

PyCHC includes features beyond certified solving, that aim at facilitating the user experience, learning and prototyping of new algorithms, and investigation of cooperative solving. Many examples can be found in PyCHC's repository.[10]

Prototyping new algorithms. PyCHC can be used for teaching and experimenting with the design of algorithms that reduce to CHC solving. With the purpose of illustrating the modelling capabilities and ease of prototyping offered by PyCHC, we have implemented the *k-liveness* algorithm [11] in Figure 4. Consider a transition system $Sys = \langle Init(X), Trans(X, X') \rangle$, where X is a set of state variables, $Init(X)$ is the initial condition, and $Trans(X, X')$ is the transition relation. The k-liveness encoding allows for checking whether every path of Sys satisfies the linear temporal logic (LTL) property $\mathbf{FG}\neg Bad$ through a series of safety checks. This is done by introducing a fresh variable k, initialised with 0 and incremented every time Bad is encountered. If, for some value $K \in \mathbb{N}$, such new transition system is safe w.r.t. the property $k < K$, then the algorithm concludes that the property $\mathbf{FG}\neg Bad$ holds on the original system, since in every path the number of occurrences of bad states is bounded by K.

This code snippet in Figure 4 shows how to implement this algorithm using PyCHC APIs. For space reasons, the snippet assumes lists of symbols `xs` and `n_xs` representing the set of variables X and X', already augmented with a variable `k` and its next-state version `n_k`, as well as formulae `init(xs)`, `trans(xs, n_xs)`, and `bad(xs)`. In lines 1–14, a CHC system is created[11] with an uninterpreted predicate `inv` on $|X| + 1$ integer variables, representing the invariant to be inferred, and with two clauses:

$$\begin{aligned} Init(X) \wedge k = 0 &\rightarrow \texttt{inv}(X, k) \\ Trans(X, X') \wedge \texttt{inv}(X, k) \wedge \begin{pmatrix} Bad(X) \rightarrow k' = k + 1 \\ \neg Bad(X) \rightarrow k' = k \end{pmatrix} &\rightarrow \texttt{inv}(X', k') \end{aligned}$$

[10] See the examples at github.com/usi-verification-and-security/pychc/examples.

[11] Shortcuts such as `Predicate`, `Apply`, and `Clause` are provided by PyCHC, while operators such as `And`, `Implies`, `Plus`, and `LT` are directly inherited from PySMT.

```
1  sys = CHCSystem(logic=QF_UFLIA) # create CHC system
2  inv = Predicate("inv", [INT]*len(xs)) # create a new predicate
3  sys.add_predicate(inv)
4  sys.add_clauses({
5      Clause(And(init, Equals(k, Int(0))), Apply(inv, xs)), # init clause
6      Clause(
7          And(
8              trans, Apply(inv, xs),
9              Implies(bad, Equals(n_k, Plus(k, Int(1)))),
10             Implies(Not(bad), Equals(n_k, k)),
11         ),
12         Apply(inv, n_xs)
13     ) # trans clause
14 })
15 solver = GolemSolver()
16 K = 1
17 while True:
18     # add new goal
19     idx = sys.add_clause(Clause(Apply(inv, xs), LT(k, Int(K))))
20     solver.load_system(sys)
21     status = solver.solve(timeout=2)
22     if status == Status.SAT: # found a model
23         try:
24             sys.validate_sat_model(solver.get_witness(), CVC5Solver())
25             return True # Property FG !bad holds
26         except PyCHCInvalidResultException as e:
27             print(e)
28             pass
29     K += 1 # increment K
30     sys.remove_clause(idx)  # remove old goal
31     if K > 10: # Give up after 10 iterations
32         break
```

Fig. 4. Prototyping a k-liveness algorithm in PyCHC.

In line 17, the code enters a loop where it checks if the safety property $k < K$ holds on the modified system for increasing values of K. This is done by adding, at each iteration, a goal clause of the form $\texttt{inv}(X, k) \wedge k \geq K \rightarrow \bot$ to the system, and invoking a CHC solver (Golem in this case). If the solver returns UNSAT, or times out, then the loop continues with the next value of K, after removing the previous goal clause (line 30). If, instead, the solver returns SAT (shorthand for satisfiable), then the system satisfies the property: before returning, however, we can validate the invariant returned by the solver using an independent SMT solver (CVC5 in this case). If anything goes wrong during validation, an exception is raised and the files with the failing clause/proof are made available.

In our example, we limit the number of iterations to 10 (line 31) since k-liveness is an incomplete algorithm for infinite-state systems, i.e., it cannot detect that the property does not hold. Other algorithms for validating liveness properties, such as the reduction of liveness to safety, can be prototyped in a similar way.

Cooperative solving. PyCHC offers portfolio methods for invoking multiple CHC solvers in parallel and returning the first conclusive answer obtained. The ability to easily access the model produced by a CHC solver, and to modify the CHC system accordingly, makes PyCHC a suitable framework for experimenting with new strategies for incremental and cooperative CHC solving. For example, after a

CHC solver returns a satisfiable answer, the model can be used to strengthen the original CHC system, by adding the interpretation of the predicates as additional premises to the clauses. Specifically, given a valid model $\mathcal{M}$ for a system $\mathcal{S}$, PyCHC offers an API to strengthen a clause $\varphi \wedge P_1 \wedge \dots P_n \to H$ in $\mathcal{S}$ as

$$\varphi \wedge \begin{pmatrix} P_1 \wedge \dots \wedge P_n \wedge \\ \mathcal{M}(P_1) \wedge \dots \wedge \mathcal{M}(P_n) \wedge \mathcal{M}(H) \end{pmatrix} \to H$$

This can help to solve new goals for the same system $\mathcal{S}$ more efficiently, possibly by other CHC solvers. Examples on combining portfolio solving with invariant sharing between different CHC solvers can be found in the PyCHC's repository.

5 Evaluation

PyCHC's GitHub repository includes several examples demonstrating the framework's modelling functionalities, design support for new algorithms, portfolio, and cooperative solving.

In order to evaluate PyCHC's capabilities, we considered the LIA-Lin and LIA categories of the 2025 edition of CHC-COMP[12], totalling 1312+1283 benchmarks, respectively. We ran Eldarica (v2.2.1), Golem (v0.9.1), and Z3-Spacer (v4.15.0) via PyCHC to solve all benchmarks and validate the produced results. Under a solving time limit of 30 seconds, Eldarica solved 793 SAT and 578 UNSAT benchmarks; Golem solved 1379 SAT and 814 UNSAT benchmarks; Z3-Spacer solved 1565 SAT and 827 UNSAT benchmarks.

We ran the validation on all satisfiable answers using cvc5 as SMT solver under a time limit of 3 minutes per clause. In this pipeline, PyCHC needs to parse the input system, parse the produced CHC model, produce and serialise the SMT queries (in addition to orchestrating the invocation of the CHC and SMT solver). The aggregated time on solved benchmarks is distributed as follows: 97.5% solving, 0.9% PyCHC, 1.6% validation for Eldarica; 69.5% solving, 9.7% PyCHC, 20.8% validation for Golem; 76.4% solving, 2.3% PyCHC, 21.3% validation for Z3-Spacer.[13] The overhead introduced by PyCHC is clearly dominated by the time spent in the actual CHC solving and SMT validation.

Unsatisfiable answers produced by Golem were validated by running Carcara on the produced Alethe proofs. In this pipeline, PyCHC does not introduce any overhead because it simply orchestrates the invocation of Carcara on the produced proof. On the solved benchmarks, the aggregated time is distributed in 87.7% solving, and 12.3% validation.

Among the CHC-COMP'25 benchmarks, we found 12 benchmarks where Eldarica returns an ill-typed model, and 7 benchmarks where Z3-Spacer returns an invalid model. We also considered the bugs discovered in [33] and analyzed the benchmarks reported in the corresponding GitHub issues, some of which have been marked closed by the solvers' developers. During our experiments, we

[12] Benchmarks available at github.com/chc-comp/chc-comp25-benchmarks.
[13] Excluding 14 Z3-Spacer models where validation timed-out.

Table 1. Issues found and their status.

Solver	Bug Effect	Current Status
Eldarica	Ill-typed model produced[15]	open
Golem	Error in Alethe proof production[16]	open
Z3-Spacer	Invalid model produced[17]	open
cvc5	Crash in proof production[18]	fixed
Golem	Crash in model production[19]	fixed
Golem	Syntactically malformed model produced[20]	fixed
OpenSMT	Crash in sort inference[21]	fixed

also found new issues regarding the unsatisfiability proofs produced by Golem that were undetected in the experiments conducted in [34].

Table 1 summarises the issues we found in our evaluation and their current status. We provide test cases[14] to reproduce each reported bug. For each "open" issue from Table 1, we provide a test case showing that PyCHC correctly raises an exception when validating the produced witness. For each "fixed" issue, we provide a test case where two versions of the corresponding tool are used, showing that PyCHC identifies the issue when using the old version, and successfully validates the produced witness when using a newer release. These test cases serve as a demonstration that PyCHC easily allows for running different versions of the same tool and comparing their results.

6 Conclusions

We presented PyCHC, a novel framework for certified Horn solving and CHC-based design. The framework offers easy-to-use APIs to create and manipulate CHC systems programmatically, and solve them using different backend solvers. Certification of both satisfiable and unsatisfiable answers is natively supported, using independent SMT solvers and proof checkers. We showed how PyCHC can be used to prototype new algorithms that reduce to CHC solving, and new strategies for cooperative solving. We also used PyCHC to validate the results of the Eldarica, Golem, and Z3-Spacer solvers on CHC-COMP benchmarks, finding many issues and tracking their status across different tool versions.

As future work, we plan to extend PyCHC with support for additional solvers and proof checkers. We also plan to extend the supported theories beyond LRA and LIA to include arrays and algebraic data types.

[14] See github.com/usi-verification-and-security/pychc/tests/test_expected_bugs.py
[15] See github.com/uuverifiers/eldarica/issues/51.
[16] See github.com/usi-verification-and-security/golem/issues/161.
[17] See github.com/Z3Prover/z3/issues/6716.
[18] See github.com/cvc5/cvc5/issues/9770.
[19] See github.com/usi-verification-and-security/golem/issues/29.
[20] See github.com/usi-verification-and-security/golem/issues/27.
[21] See github.com/usi-verification-and-security/opensmt/issues/613.

Acknowledgments. We thank the anonymous reviewers for their valuable feedback and suggestions. This work was partially conducted in the scope of the “Formal Reasoning on Neural Networks" project funded by the Hasler Foundation, Switzerland, and the Swiss National Science Foundation grant 200021-236601 “Cross Theory Rigorous Program Verification Using Constrained Horn Clauses".

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

Data-Availability Statement. A self-contained artifact able to reproduce all the claims made in this paper is available at https://doi.org/10.5281/zenodo.19738165.

References

1. Andreotti, B., Lachnitt, H., Barbosa, H.: Carcara: An Efficient Proof Checker and Elaborator for SMT Proofs in the Alethe Format. In: Proceedings of the 29th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'23). pp. 367–386 (2023). https://doi.org/10.1007/978-3-031-30823-9_19
2. Baek, S., Carneiro, M., Heule, M.J.H.: A Flexible Proof Format for SAT Solver-Elaborator Communication. In: Proceedings of the 27th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'21). pp. 59–75 (2021). https://doi.org/10.1007/978-3-030-72016-2_4
3. Baier, D., Beyer, D., Friedberger, K.: JavaSMT 3: Interacting with SMT Solvers in Java. In: Proceedings of the 33rd International Conference on Computer Aided Verification (CAV'21). pp. 195–208 (2021). https://doi.org/10.1007/978-3-030-81688-9_9
4. Barbosa, H., Barrett, C., Brain, M., Kremer, G., Lachnitt, H., Mann, M., Mohamed, A., Mohamed, M., Niemetz, A., Nötzli, A., Ozdemir, A., Preiner, M., Reynolds, A., Sheng, Y., Tinelli, C., Zohar, Y.: cvc5: A Versatile and Industrial-Strength SMT Solver. In: Proceedings of the 28th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'22). pp. 415–442 (2022). https://doi.org/10.1007/978-3-030-99524-9_24
5. Barrett, C., Fontaine, P., Tinelli, C.: The SMT-LIB Standard: Version 2.7. https://smt-lib.org/papers/smt-lib-reference-v2.7-r2025-07-07.pdf (2025), Accessed: 2026-01-18
6. Bjørner, N., Gurfinkel, A., McMillan, K., Rybalchenko, A.: Horn Clause Solvers for Program Verification. Fields of Logic and Computation II **9300**, pp. 24–51 (2015). https://doi.org/10.1007/978-3-319-23534-9_2
7. Blicha, M., Britikov, K., Sharygina, N.: Golem: a Flexible and Efficient Solver for Constrained Horn Clauses. Formal Methods in System Design (FMSD) **67**(2), pp. 143–160 (2025). https://doi.org/10.1007/s10703-025-00470-9
8. Bouton, T., Caminha B. de Oliveira, D., Déharbe, D., Fontaine, P.: veriT: An Open, Trustable and Efficient SMT-Solver. In: Proceedings of the 22nd International Conference on Automated Deduction (CADE'09). pp. 151–156 (2009). https://doi.org/10.1007/978-3-642-02959-2_12
9. Bruttomesso, R., Pek, E., Sharygina, N., Tsitovich, A.: The OpenSMT Solver. In: Proceedings of the 16th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'10). pp. 150–153 (2010). https://doi.org/10.1007/978-3-642-12002-2_12

10. Champion, A., Kobayashi, N., Sato, R.: HoIce: An ICE-Based Non-linear Horn Clause Solver. In: Proceedings of the 16th Asian Symposium on Programming Languages and Systems (APLAS'18). pp. 146–156 (2018). https://doi.org/10.1007/978-3-030-02768-1_8
11. Claessen, K., Sörensson, N.: A Liveness Checking Algorithm that Counts. In: Proceedings of the 12th Conference on Formal Methods in Computer-Aided Design (FMCAD'12). pp. 52–59 (2012), https://ieeexplore.ieee.org/document/6462555
12. Cruz-Filipe, L., Heule, M.J.H., Hunt, W.A., Kaufmann, M., Schneider-Kamp, P.: Efficient Certified RAT Verification. In: Proceedings of the 26th International Conference on Automated Deduction (CADE'17). pp. 220–236 (2017). https://doi.org/10.1007/978-3-319-63046-5_14
13. Ernst, G., Morales, J.F., Bajczi, L.: CHC-COMP 2025 Extended Report. https://chc-comp.github.io/2025/CHC-COMP 2025 Extended Report - HCVS.pdf (2025), Accessed: 2026-01-17
14. Fedyukovich, G., Kaufman, S.J., Bodík, R.: Sampling Invariants from Frequency Distributions. In: Proceedings of the 17th Conference on Formal Methods in Computer-Aided Design (FMCAD'17). pp. 100–107 (2017). https://doi.org/10.23919/FMCAD.2017.8102247
15. Gario, M., Micheli, A.: PySMT: a Solver-agnostic Library for Fast Prototyping of SMT-based Algorithms. In: Proceedings of the 13th International Workshop on Satisfiability Modulo Theories (SMT'15). pp. 1–10 (2015), https://github.com/pysmt/pysmt
16. Gurfinkel, A., Bjørner, N.: The Science, Art, and Magic of Constrained Horn Clauses. In: Proceedings of the 21st International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC'19). pp. 6–10 (2019). https://doi.org/10.1109/SYNASC49474.2019.00010
17. Gurfinkel, A., Kahsai, T., Komuravelli, A., Navas, J.A.: The SeaHorn Verification Framework. In: Proceedings of the 27th International Conference on Computer Aided Verification (CAV'15). pp. 343–361 (2015). https://doi.org/10.1007/978-3-319-21690-4_20
18. Gurfinkel, A., Shoham, S., Meshman, Y.: SMT-based Verification of Parameterized Systems. In: Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE'16). pp. 338–348. (2016). https://doi.org/10.1145/2950290.2950330
19. Heule, M.J.H., Hunt, W.A., Wetzler, N.: Verifying Refutations with Extended Resolution. In: Proceedings of the 24th International Conference on Automated Deduction (CADE'13). pp. 345–359 (2013). https://doi.org/10.1007/978-3-642-38574-2_24
20. Heule, M.J., Hunt, W.A., Wetzler, N.: Trimming While Checking Clausal Proofs. In: Proceedings of the 13th Conference on Formal Methods in Computer-Aided Design (FMCAD'13). pp. 181–188 (2013). https://doi.org/10.1109/FMCAD.2013.6679408
21. Hoenicke, J., Schindler, T.: A Simple Proof Format for SMT. In: Proceedings of the 20th International Workshop on Satisfiability Modulo Theories (SMT'22). pp. 54–70 (2022), https://ceur-ws.org/Vol-3185/paper9527.pdf
22. Hojjat, H., Rümmer, P.: The Eldarica Horn Solver. In: Proceedings of the 18th Conference on Formal Methods in Computer-Aided Design (FMCAD'18). pp. 1–7 (2018). https://doi.org/10.23919/FMCAD.2018.8603013
23. Hyvärinen, A.E.J., Marescotti, M., Alt, L., Sharygina, N.: OpenSMT2: An SMT Solver for Multi-core and Cloud Computing. In: Proceedings of the 19th Interna-

tional Conference on Theory and Applications of Satisfiability Testing (SAT'16). pp. 547–553 (2016). https://doi.org/10.1007/978-3-319-40970-2_35
24. Itzhaky, S., Shoham, S., Vizel, Y.: Hyperproperty Verification as CHC Satisfiability. In: Proceedings of the 33rd European Symposium on Programming (ESOP'24). pp. 212–241 (2024). https://doi.org/10.1007/978-3-031-57267-8_9
25. Jonáš, M., Bobot, F., Déharbe, D., Winterer, D.: SMT-COMP 2025 Experimental Model Validation Track. https://smt-comp.github.io/2025/model (2025), Accessed: 2026-01-17
26. Kahsai, T., Rümmer, P., Sanchez, H., Schäf, M.: JayHorn: A Framework for Verifying Java Programs. In: Proceedings of the 28th International Conference on Computer Aided Verification (CAV'16). pp. 352–358 (2016). https://doi.org/10.1007/978-3-319-41528-4_19
27. Kim, J., Hu, Q., D'Antoni, L., Reps, T.W.: Semantics-guided Synthesis. Proceedings of the ACM on Programming Languages (PACMPL) **5**(POPL), pp. 1–32 (2021). https://doi.org/10.1145/3434311
28. Komuravelli, A., Gurfinkel, A., Chaki, S.: SMT-Based Model Checking for Recursive Programs. Formal Methods in System Design (FMSD) **48**(3), pp. 175–205 (2016). https://doi.org/10.1007/s10703-016-0249-4
29. Matsushita, Y., Tsukada, T., Kobayashi, N.: RustHorn: CHC-Based Verification for Rust Programs. ACM Transactions on Programming Languages and Systems (TOPLAS) **43**(4), pp. 1–54 (2021). https://doi.org/10.1145/3462205
30. de Moura, L., Bjørner, N.: Proofs and Refutations, and Z3. In: Proceedings of the 7th International Workshop on the Implementation of Logics (IWIL'08). pp. 123–132 (2008), https://ceur-ws.org/Vol-418/paper10.pdf
31. de Moura, L., Bjørner, N.: Z3: An Efficient SMT Solver. In: Proceedings of the 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'08). pp. 337–340 (2008). https://doi.org/10.1007/978-3-540-78800-3_24
32. Otoni, R., Blicha, M., Eugster, P., Hyvärinen, A.E.J., Sharygina, N.: Theory-Specific Proof Steps Witnessing Correctness of SMT Executions. In: Proceedings of the 58th ACM/IEEE Design Automation Conference (DAC'21). pp. 541–546 (2021). https://doi.org/10.1109/DAC18074.2021.9586272
33. Otoni, R., Blicha, M., Eugster, P., Sharygina, N.: Validation of CHC Satisfiability with ATHENA. Formal Aspects of Computing (FAC) **37**(4), pp. 1–20 (2025). https://doi.org/10.1145/3716505
34. Otoni, R., Blicha, M., Rivera, M.B., Eugster, P., Kofroň, J., Sharygina, N.: Unsatisfiability Proofs for Horn Solving. In: Proceedings of the 31st International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'25). pp. 67–87 (2025). https://doi.org/10.1007/978-3-031-90653-4_4
35. Otoni, R., Marescotti, M., Alt, L., Eugster, P., Hyvärinen, A., Sharygina, N.: A Solicitous Approach to Smart Contract Verification. ACM Transactions on Privacy and Security (TOPS) **26**(2), pp. 1–28 (2023). https://doi.org/10.1145/3564699
36. Pereira, J.M., Marques, F., Adão, P., Hara, H.R.A.E., Andrès, L., Carcano, A., Chambart, P., Maksimovic, P., Santos, N., Santos, J.F.: smt.ml: A Multi-Backend Frontend for SMT Solvers in OCaml. In: Proceedings of the 32nd International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'26). pp. 23–44 (2026). https://doi.org/10.1007/978-3-032-22752-2_2
37. Rothenberg, B., Grumberg, O., Vizel, Y., Singher, E.: Condition Synthesis Realizability via Constrained Horn Clauses. In: Proceedings of the 15th NASA Formal Methods Symposium (NFM'23). pp. 380–396, (2023). https://doi.org/10.1007/978-3-031-33170-1_23

38. Rümmer, P., Hojjat, H., Kuncak, V.: On Recursion-Free Horn Clauses and Craig Interpolation. Formal Methods in System Design (FMSD) **47**(1), pp. 1–25 (2015). https://doi.org/10.1007/s10703-014-0219-7
39. Schurr, H.J., Fleury, M., Barbosa, H., Fontaine, P.: Alethe: Towards a Generic SMT Proof Format. In: Proceedings of the 7th Workshop on Proof eXchange for Theorem Proving (PxTP'21). pp. 49–54 (2021). https://doi.org/10.4204/EPTCS.336
40. Sinz, C., Biere, A.: Extended Resolution Proofs for Conjoining BDDs. In: Proceedings of the 1st International Symposium on Computer Science in Russia (CSR'06). pp. 600–611 (2006). https://doi.org/10.1007/11753728_60
41. Stump, A., Oe, D., Reynolds, A., Hadarean, L., Tinelli, C.: SMT Proof Checking Using a Logical Framework. Formal Methods in System Design (FMSD) **42**(1), pp. 91–118 (2013). https://doi.org/10.1007/s10703-012-0163-3
42. Tsukada, T., Unno, H.: Inductive Approach to Spacer. Proceedings of the ACM on Programming Languages (PACMPL) **8**(PLDI), pp. 1979–2002 (2024). https://doi.org/10.1145/3656457
43. Vediramana Krishnan, H.G., Chen, Y., Shoham, S., Gurfinkel, A.: Global Guidance for Local Generalization in Model Checking. Formal Methods in System Design (FMSD) **63**(1), pp. 81–109 (2024). https://doi.org/10.1007/s10703-023-00412-3
44. Wetzler, N., Heule, M.J.H., Hunt, W.A.: DRAT-trim: Efficient Checking and Trimming Using Expressive Clausal Proofs. In: Proceedings of the 17th International Conference on Theory and Applications of Satisfiability Testing (SAT'14). pp. 422–429 (2014). https://doi.org/10.1007/978-3-319-09284-3_31